How Vectorization Works: Parallel vs. Sequential Processing

Vectorization feels like a "magic trick" because it fundamentally changes how your computer performs calculations, leveraging modern hardware for massive speed gains.

Non-Vectorized (Sequential) Processing 🚶

- A standard `for` loop executes **sequentially**.
- This means it processes one instruction at a time, one after another. If you need to perform 16 calculations, the computer does the first one, then the second, then the third, and so on, taking 16 distinct time steps.

Vectorized (Parallel) Processing 🚀

- Vectorized operations, like those in NumPy, use **parallel processing**.
- The computer's hardware (CPU or GPU) is designed to perform the same operation on all elements of an array **at the same time, in a single step**.
- For example, to multiply two vectors with 16 elements, the hardware can perform all 16 multiplications simultaneously. It then uses specialized hardware to sum the results very efficiently.
- This parallel execution is why vectorization is dramatically faster, especially for large datasets. It can be the difference between an algorithm finishing in minutes versus taking many hours.

Vectorizing the Gradient Descent Update

This principle can be directly applied to the gradient descent update rule for the weight parameters.

- **Goal:** Update each parameter w_j according to the rule: $w_j := w_j - \alpha \cdot d_j$ (where d_j is the derivative for that parameter).

Non-Vectorized (For Loop) Approach

You would loop through each parameter and update it individually.

```
# Slower, sequential update
for j in range(n):
  w[j] = w[j] - alpha * d[j]
```

Vectorized Approach ✅

You treat `w` and `d` as vectors (or NumPy arrays) and perform the update in a single line of code.

```
# Faster, parallel update
w = w - alpha * d
```

Behind the scenes, the computer performs all the subtractions and multiplications for every element of the `w` vector at the same time.